

```

xx=x[i];
t=b=1.0;
s=q[n-1];
for (k=n-1;k>0;k--) {
    b=c[k]+xx*b;
    s += q[k-1]*b;
    t=xx*t+b;
}
w[i]=s/t;
}
}
}

```

is synthetically divided,
matrix-multiplied by the right-hand side,
and supplied with a denominator.

2.8.2 Toeplitz Matrices

An $N \times N$ Toeplitz matrix is specified by giving $2N - 1$ numbers R_k , where the index k ranges over $k = -N + 1, \dots, -1, 0, 1, \dots, N - 1$. Those numbers are then emplaced as matrix elements constant along the (upper-left to lower-right) diagonals of the matrix:

$$\begin{bmatrix} R_0 & R_{-1} & R_{-2} & \cdots & R_{-(N-2)} & R_{-(N-1)} \\ R_1 & R_0 & R_{-1} & \cdots & R_{-(N-3)} & R_{-(N-2)} \\ R_2 & R_1 & R_0 & \cdots & R_{-(N-4)} & R_{-(N-3)} \\ \cdots & & & & & \\ R_{N-2} & R_{N-3} & R_{N-4} & \cdots & R_0 & R_{-1} \\ R_{N-1} & R_{N-2} & R_{N-3} & \cdots & R_1 & R_0 \end{bmatrix} \quad (2.8.8)$$

The linear Toeplitz problem can thus be written as

$$\sum_{j=0}^{N-1} R_{i-j} x_j = y_i \quad (i = 0, \dots, N-1) \quad (2.8.9)$$

where the x_j 's, $j = 0, \dots, N-1$, are the unknowns to be solved for.

The Toeplitz matrix is symmetric if $R_k = R_{-k}$ for all k . Levinson [4] developed an algorithm for fast solution of the symmetric Toeplitz problem, by a *bordering method*, that is, a recursive procedure that solves the $(M+1)$ -dimensional Toeplitz problem

$$\sum_{j=0}^M R_{i-j} x_j^{(M)} = y_i \quad (i = 0, \dots, M) \quad (2.8.10)$$

in turn for $M = 0, 1, \dots$ until $M = N-1$, the desired result, is finally reached. The vector $x_j^{(M)}$ is the result at the M th stage and becomes the desired answer only when $N-1$ is reached.

Levinson's method is well documented in standard texts (e.g., [5]). The useful fact that the method generalizes to the *nonsymmetric* case seems to be less well known. At some risk of excessive detail, we therefore give a derivation here, due to G.B. Rybicki.

In following a recursion from step M to step $M+1$ we find that our developing solution $x^{(M)}$ changes in this way:

$$\sum_{j=0}^M R_{i-j} x_j^{(M)} = y_i \quad i = 0, \dots, M \quad (2.8.11)$$

becomes

$$\sum_{j=0}^M R_{i-j} x_j^{(M+1)} + R_{i-(M+1)} x_{M+1}^{(M+1)} = y_i \quad i = 0, \dots, M+1 \quad (2.8.12)$$

By eliminating y_i we find

$$\sum_{j=0}^M R_{i-j} \left(\frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}} \right) = R_{i-(M+1)} \quad i = 0, \dots, M \quad (2.8.13)$$

or by letting $i \rightarrow M - i$ and $j \rightarrow M - j$,

$$\sum_{j=0}^M R_{j-i} G_j^{(M)} = R_{-(i+1)} \quad (2.8.14)$$

where

$$G_j^{(M)} \equiv \frac{x_{M-j}^{(M)} - x_{M-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \quad (2.8.15)$$

To put this another way,

$$x_{M-j}^{(M+1)} = x_{M-j}^{(M)} - x_{M+1}^{(M+1)} G_j^{(M)} \quad j = 0, \dots, M \quad (2.8.16)$$

Thus, if we can use recursion to find the order M quantities $x^{(M)}$ and $G^{(M)}$ and the single order $M + 1$ quantity $x_{M+1}^{(M+1)}$, then all of the other $x_j^{(M+1)}$'s will follow. Fortunately, the quantity $x_{M+1}^{(M+1)}$ follows from equation (2.8.12) with $i = M + 1$,

$$\sum_{j=0}^M R_{M+1-j} x_j^{(M+1)} + R_0 x_{M+1}^{(M+1)} = y_{M+1} \quad (2.8.17)$$

For the unknown order $M + 1$ quantities $x_j^{(M+1)}$ we can substitute the previous order quantities in G since

$$G_{M-j}^{(M)} = \frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}} \quad (2.8.18)$$

The result of this operation is

$$x_{M+1}^{(M+1)} = \frac{\sum_{j=0}^M R_{M+1-j} x_j^{(M)} - y_{M+1}}{\sum_{j=0}^M R_{M+1-j} G_{M-j}^{(M)} - R_0} \quad (2.8.19)$$

The only remaining problem is to develop a recursion relation for G . Before we do that, however, we should point out that there are actually two distinct sets of solutions to the original linear problem for a nonsymmetric matrix, namely right-hand solutions (which we have been discussing) and left-hand solutions z_i . The formalism for the left-hand solutions differs only in that we deal with the equations

$$\sum_{j=0}^M R_{j-i} z_j^{(M)} = y_i \quad i = 0, \dots, M \quad (2.8.20)$$

Then, the same sequence of operations on this set leads to

$$\sum_{j=0}^M R_{i-j} H_j^{(M)} = R_{i+1} \quad (2.8.21)$$

where

$$H_j^{(M)} \equiv \frac{z_{M-j}^{(M)} - z_{M-j}^{(M+1)}}{z_{M+1}^{(M+1)}} \quad (2.8.22)$$

(compare with 2.8.14 – 2.8.15). The reason for mentioning the left-hand solutions now is that, by equation (2.8.21), the H_j 's satisfy exactly the same equation as the x_j 's except for the substitution $y_i \rightarrow R_{i+1}$ on the right-hand side. Therefore we can quickly deduce from equation (2.8.19) that

$$H_{M+1}^{(M+1)} = \frac{\sum_{j=0}^M R_{M+1-j} H_j^{(M)} - R_{M+2}}{\sum_{j=0}^M R_{M+1-j} G_{M-j}^{(M)} - R_0} \quad (2.8.23)$$

By the same token, G satisfies the same equation as z , except for the substitution $y_i \rightarrow R_{-(i+1)}$. This gives

$$G_{M+1}^{(M+1)} = \frac{\sum_{j=0}^M R_{j-M-1} G_j^{(M)} - R_{-M-2}}{\sum_{j=0}^M R_{j-M-1} H_{M-j}^{(M)} - R_0} \quad (2.8.24)$$

The same “morphism” also turns equation (2.8.16), and its partner for z , into the final equations

$$\begin{aligned} G_j^{(M+1)} &= G_j^{(M)} - G_{M+1}^{(M+1)} H_{M-j}^{(M)} \\ H_j^{(M+1)} &= H_j^{(M)} - H_{M+1}^{(M+1)} G_{M-j}^{(M)} \end{aligned} \quad (2.8.25)$$

Now, starting with the initial values

$$x_0^{(0)} = y_0/R_0 \quad G_0^{(0)} = R_{-1}/R_0 \quad H_0^{(0)} = R_1/R_0 \quad (2.8.26)$$

we can recurse away. At each stage M we use equations (2.8.23) and (2.8.24) to find $H_{M+1}^{(M+1)}$, $G_{M+1}^{(M+1)}$, and then equation (2.8.25) to find the other components of $H^{(M+1)}$, $G^{(M+1)}$. From there the vectors $x^{(M+1)}$ and/or $z^{(M+1)}$ are easily calculated.

The program below does this. It incorporates the second equation in (2.8.25) in the form

$$H_{M-j}^{(M+1)} = H_{M-j}^{(M)} - H_{M+1}^{(M+1)} G_j^{(M)} \quad (2.8.27)$$

so that the computation can be done “in place.”

Notice that the above algorithm fails if $R_0 = 0$. In fact, because the bordering method does not allow pivoting, the algorithm will fail if any of the diagonal principal minors of the original Toeplitz matrix vanish. (Compare with discussion of the tridiagonal algorithm in §2.4.) If the algorithm fails, your matrix is not necessarily singular — you might just have to solve your problem by a slower and more general algorithm such as LU decomposition with pivoting.

The routine that implements equations (2.8.23) – (2.8.27) is also due to Rybicki. Note that the routine's $r[n-1+j]$ is equal to R_j above, so that subscripts on the r array vary from 0 to $2N - 2$.

`toeplz.h` `void toeplz(VecDoub_I &r, VecDoub_O &x, VecDoub_I &y)`

Solves the Toeplitz system $\sum_{j=0}^{N-1} R_{(N-1+i-j)} x_j = y_i$ ($i = 0, \dots, N-1$). The Toeplitz matrix need not be symmetric. $y[0..n-1]$ and $r[0..2n-2]$ are input arrays; $x[0..n-1]$ is the output array.

```
{
    Int j,k,m,m1,m2,n1,n=y.size();
    Doub pp,pt1,pt2,qq,qt1,qt2,sd,sgd,sgn,shn,sxn;
    n1=n-1;
```

```

if (r[n1] == 0.0) throw("toeplz-1 singular principal minor");
x[0]=y[0]/r[n1];           Initialize for the recursion.
if (n1 == 0) return;
VecDoub g(n1),h(n1);
g[0]=r[n1-1]/r[n1];
h[0]=r[n1+1]/r[n1];
for (m=0;m<n;m++) {        Main loop over the recursion.
    m1=m+1;
    sxn = -y[m1];           Compute numerator and denominator for x from eq.
    sd = -r[n1];             (2.8.19),
    for (j=0;j<m+1;j++) {
        sxn += r[n1+m1-j]*x[j];
        sd += r[n1+m1-j]*g[m-j];
    }
    if (sd == 0.0) throw("toeplz-2 singular principal minor");
    x[m1]=sxn/sd;           whence x.
    for (j=0;j<m+1;j++)     Eq. (2.8.16).
        x[j] -= x[m1]*g[m-j];
    if (m1 == n1) return;
    sgn = -r[n1-m1-1];      Compute numerator and denominator for G and H,
    shn = -r[n1+m1+1];      eqs. (2.8.24) and (2.8.23),
    sgd = -r[n1];
    for (j=0;j<m+1;j++) {
        sgn += r[n1+j-m1]*g[j];
        shn += r[n1+m1-j]*h[j];
        sgd += r[n1+j-m1]*h[m-j];
    }
    if (sgd == 0.0) throw("toeplz-3 singular principal minor");
    g[m1]=sgn/sgd;          whence G and H.
    h[m1]=shn/sd;
    k=m;
    m2=(m+2) >> 1;
    pp=g[m1];
    qq=h[m1];
    for (j=0;j<m2;j++) {
        pt1=g[j];
        pt2=g[k];
        qt1=h[j];
        qt2=h[k];
        g[j]=pt1-pp*qt2;
        g[k]=pt2-pp*qt1;
        h[j]=qt1-qq*pt2;
        h[k--]=qt2-qq*pt1;
    }
}
}                               Back for another recurrence.
throw("toeplz - should not arrive here!");
}

```

If you are in the business of solving *very* large Toeplitz systems, you should find out about so-called “new, fast” algorithms, which require only on the order of $N(\log N)^2$ operations, compared to N^2 for Levinson’s method. These methods are too complicated to include here. Papers by Bunch [6] and de Hoog [7] will give entry to the literature.

CITED REFERENCES AND FURTHER READING:

- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 5 [also treats some other special forms].
- Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), §19.[1]
- Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).[2]